

OPERATING SYSTEMS – PROCESS CONTROL

Task 1 – Parent and Child Process Using fork()

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process execution information.

Execution Evidence

The following terminal output records the compilation and execution of process_fork.c.

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution
Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Figure 1 – Terminal execution evidence supplied for the process_fork program.

Recorded Output

```
speed@DESKTOP-P4G181S:~$ nano process_fork.c
speed@DESKTOP-P4G181S:~$ gcc process_fork.c -o process_fork
speed@DESKTOP-P4G181S:~$ ./process_fork
Parent process started.Parent PID : 455Parent PPID : 323
----- PARENT PROCESS -----Parent PID : 455Child PID : 456Parent is waiting for child...----- CHILD PROCESS -----Child PID : 456Child PPID : 455Child is running...Child is terminating...Child process terminated.Parent process completed.
speed@DESKTOP-P4G181S:~$
```

```
speed@DESKTOP-P4G181S:~$ nano process_fork.c
speed@DESKTOP-P4G181S:~$ gcc process_fork.c -o process_fork
speed@DESKTOP-P4G181S:~$ ./process_fork
Parent process started.
Parent PID : 455
Parent PPID : 323

----- PARENT PROCESS -----
Parent PID : 455
Child PID : 456
Parent is waiting for child...
----- CHILD PROCESS -----
Child PID : 456
Child PPID : 455
Child is running...
Child is terminating...
Child process terminated.
Parent process completed.
speed@DESKTOP-P4G181S:~$ |
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("----- CHILD PROCESS ----- \n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is running...\n");

        sleep(5);

        printf("Child is terminating...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("----- PARENT PROCESS ----- \n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child process terminated.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}

```

2 – Process State Transition Experiment

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Execution Evidence

The following terminal output records the compilation and execution of process_states.c.

Recorded Output

```

speed@DESKTOP-P4G181S:~$ nano process_states.c
speed@DESKTOP-P4G181S:~$ gcc process_states.c -o process_states
speed@DESKTOP-P4G181S:~$ ./process_states
Parent process created.
Parent PID : 498
Child PID : 499
Parent will wait for the child.
Child process created.
Child PID : 499
Parent PID : 498
STATE 1:
RUNNING
Child is executing CPU instructions...
Child is still running...
Child is still running...
Child is

```

still running...Child is still running...Child is still running...STATE 2: WAITINGChild is sleeping for 15 seconds...STATE 3: TERMINATINGChild is exiting now.STATE 4: CHILD TERMINATEDParent detected child termination.speed@DESKTOP-P4G181S:~\$

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created.\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nSTATE 1: RUNNING\n");
        printf("Child is executing CPU instructions...\n");

        for (volatile long i = 0; i < 5000000000L; i++)
        {
            if (i % 1000000000L == 0)
                printf("Child is still running...\n");
        }

        printf("\nSTATE 2: WAITING\n");
        printf("Child is sleeping for 15 seconds...\n");

        sleep(15);

        printf("\nSTATE 3: TERMINATING\n");
        printf("Child is exiting now.\n");

        exit(0);
    }
    else
    {
        printf("Parent process created.\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("\nParent will wait for the child.\n");
        wait(NULL);

        printf("\nSTATE 4: CHILD TERMINATED\n");
        printf("Parent detected child termination.\n");
    }

    return 0;
}
```

```

speed@DESKTOP-P4G181S:~$ nano process_states.c
speed@DESKTOP-P4G181S:~$ gcc process_states.c -o process_states
speed@DESKTOP-P4G181S:~$ ./process_states
Parent process created.
Parent PID : 498
Child PID  : 499

Parent will wait for the child.
Child process created.
Child PID  : 499
Parent PID : 498

STATE 1: RUNNING
Child is executing CPU instructions...
Child is still running...
Child is still running...
Child is still running...
Child is still running...
Child is still running...

STATE 2: WAITING
Child is sleeping for 15 seconds...

STATE 3: TERMINATING
Child is exiting now.

STATE 4: CHILD TERMINATED
Parent detected child termination.

```

Observation Table

Stage	Expected/Observed State	Tool / Evidence	Observation
STATE 1	RUNNING	ps / top	Child is executing CPU instructions.
STATE 2	WAITING	ps / /proc	Child is sleeping for 15 seconds.
STATE 3	TERMINATING	Program output	Child is exiting.
STATE 4	TERMINATED	Parent output	Parent detects child termination.

Conclusion

The experiment demonstrates creation of parent and child processes using `fork()` and records the child process at different execution stages. The supplied execution output shows RUNNING, WAITING, TERMINATING, and CHILD TERMINATED stages. Linux monitoring screenshots can be inserted in the designated slots above.